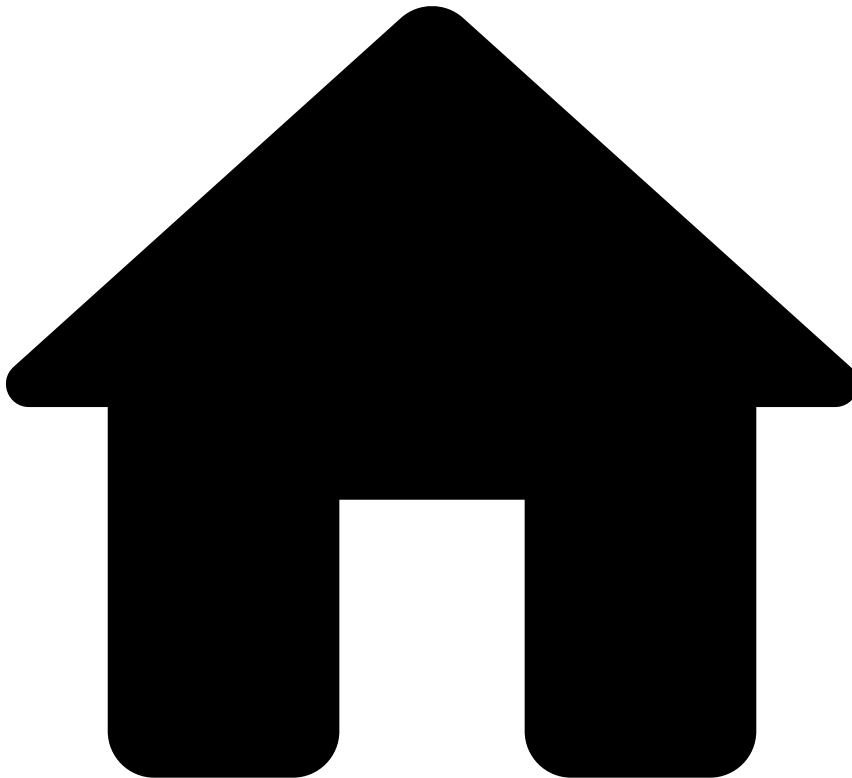


•



- [Operational Work](#)
- [Data Scientist & Fraud Strategist](#)
- [Use Models](#)
- [AutoML](#)
- AutoML Stages

On this page

AutoML Stages

Was this page helpful? 

You can use AutoML to create features and train models in an automated way. AutoML evaluates the models and recommends the most performant one. To accomplish this, the risk engine platform (Pulse) performs a number of operations (i.e. jobs) described in the following sections.

AutoML process

AutoML relies on a smart engine that generates features automatically based on prior data science knowledge. In addition to generating features, the AutoML also performs the following tasks:

1. Uses the most important features to train multiple models.
2. Recommends the most suitable model for the use case. You can then integrate this model into the Pulse workflow to score transactions in real time.

To give you a more granular view about what happens, the following figure summarizes the key stages of AutoML:

To run AutoML, you need to provide:

- A data source with historical data.
- A [semantic file](#) that maps the input data fields to semantic tags.



danger

Environment to run AutoML: Since the AutoML computation is quite resource-intensive for the system, Feedzai recommends running AutoML in a staging environment dedicated to the data science work (i.e. typically a Data Science environment). This is important to ensure that the production environment is not compromised by this functionality. Once you achieve results that meet your business needs, you can [migrate](#) the AutoML model and plan to the production environment and [integrate these new resources into the runtime workflow](#).

Semantic file

To give meaning to input data fields in an AutoML run, you need to have a semantic file that maps the data fields of the input data source to semantic tags. The semantic tags translate the meaning of the fields, thus providing the only portion of human knowledge that is required to power AutoML.

[Download](#) the standard semantic files that apply to your use case.



info

Importance of the semantic tags: The number and quality of the features generated by AutoML depend significantly on how the fields are tagged in the semantic file. Tagging the input data fields incorrectly may not produce meaningful features (or features at all). Note that the quality of models depends on features, and consequently on model results (e.g. the ROC curve and metrics depend heavily on the tagging quality).

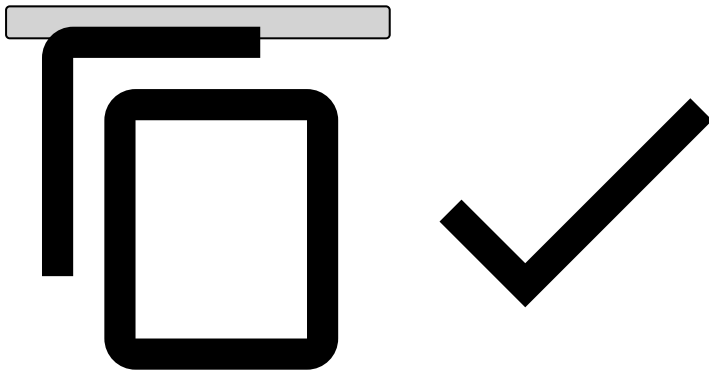
The following example is an excerpt of a semantic file:

```
[  
{
```

```

"@type": "simple",
"tags": [
  "amount"
],
"fieldDesc": "amount"
},
{
"@type": "simple",
"tags": [
  "grouping-entity",
  "entity"
],
"fieldDesc": "acceptor_id"
},
{
"@type": "simple",
"tags": [
  "entity",
  "country",
  "location"
],
"fieldDesc": "acceptor_country_code"
}
...

```



The `fieldDesc` corresponds to the tagged field, and the `tag` corresponds to the semantic meaning given to that field.

Each stage of the AutoML process is further explained in the following sections.

Feature engineering

Feature engineering is the process of applying domain knowledge to derive new features from existing data. These features in Pulse are represented by maps and metric operators in a plan. During the AutoML computation, Pulse generates a plan with these operators.

AutoML has a list of data field transformations defined according to Feedzai's experience in the fraud-fighting domain. Those transformations are based on the [semantics](#) of the data fields, combining fields with specific meaning into features. For example, if a data source has two location data fields, it is useful to calculate the distance between the two. This means that the AutoML has a transformation that computes the distance between two location fields.

To allow AutoML to apply those transformations, when defining an AutoML run you need to provide a semantic file that maps the input data fields to semantic tags. For example, the `customer_name` field can be tagged with the 'name' tag. This way, AutoML knows the meaning of this input field, and it can compute the number of words in that field, which can be useful to fight fraud.

Without tagging the input data fields in a semantic file, AutoML cannot generate features. The quality of the models generated by AutoML relies heavily on the quality of tagging.

In summary, AutoML follows these steps to create features:

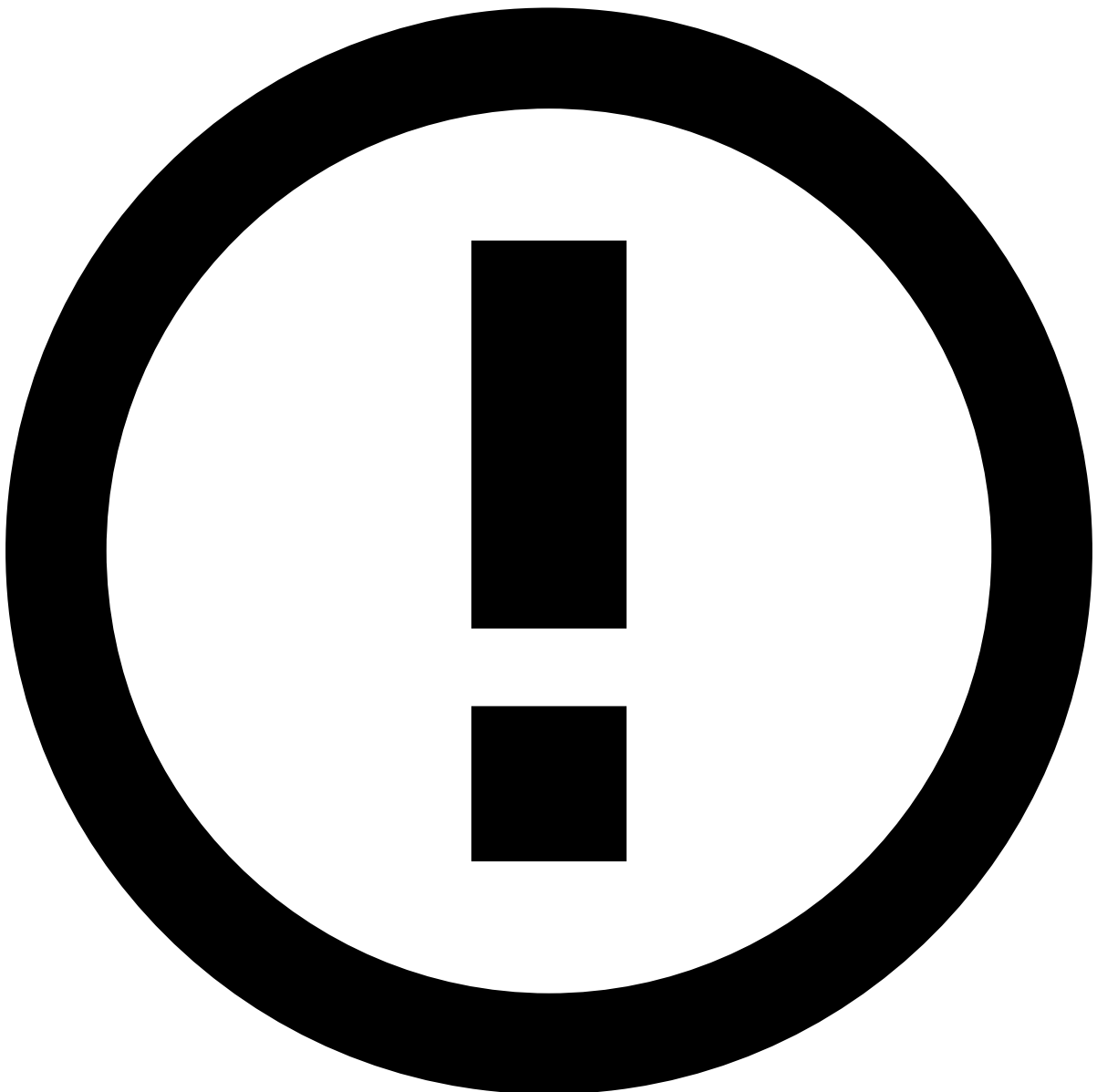
1. Looks at the semantic file and maps the data fields to their meaning.

2. Based on the data fields' meaning and the list of transformations, AutoML creates a plan containing map and metric operators (see Features Plan in the [AutoML figure](#)).
3. Executes the plan on the input data source, and saves the result in a features data source that contains the data enriched with the new data fields.

Dataset split

When setting up an AutoML run, you must define which subsets of the input data source will be used for training and validation. Optionally, you can also choose to have a test dataset to simulate a blind evaluation of the best model found.

AutoML creates the training, validation and the optional test data sources based on the defined time ranges (see Split Plan in the [AutoML figure](#)).



info

Planning training, validation and test data sources: To avoid under- or overfitting, plan the time ranges carefully. Ensure that the resulting datasets contain enough data for their purpose. A typical dataset split is 50% for training, 30% for

validation and 20% for evaluation, as long as the data source size is sufficient for that. The time windows need to be in the past and follow the order training-validation-evaluation.

If you don't define a test dataset, AutoML uses the validation dataset for the final evaluation of the best model found. Since the validation dataset was used for finding the best model, it is not an optimal solution to use it to test the data. Therefore, you should use a test dataset.

Feature selection

In this step, AutoML evaluates which features will better contribute to the model performance. Pulse uses Gradient Boosted Trees to evaluate the features over the training dataset, and ranks the features accordingly.

Then Pulse reduces the features to a subset that has the most impact in performance, keeping only a subset of the features for the model training step. The features that AutoML keeps contribute with more than 90% of the importance rating.

Model training and evaluation

After selecting the most important features, AutoML creates a hyperparameter tuning job to train and evaluate models. Hyperparameter tuning is the process of fine-tuning parameters to obtain the optimal set of parameters for a specific use case. The risk engine generates the performance statistics for a number of different algorithms, testing multiple parameter combinations. This is done based on a grid search that iterates over multiple parameters, a method also known as parameter sweep. Hyperparameter tuning jobs distribute the model training and evaluation in the Spark cluster.

Finally, AutoML compares the models based on the metric that you selected to be optimized, and recommends the best model.

Next step

Now that you know how AutoML works, learn how to [load and prepare the data](#) that will be then used to run AutoML.